

Relatório Técnico

Sistema de Agendamento de Atendimentos — agenda Saúde

Equipe agenda Saúde — PI2 (Projeto Integrador — Ciência de Dados / Engenharia de Sistemas / TI)

Data da revisão/implantação: 07/09/2026

Escopo: Backend (Python/FastAPI + Supabase), Frontend (React 18 + TypeScript + Vite + Tailwind/shadcn-ui), Infraestrutura e Deploy

Status geral: 5 pendências estruturais resolvidas 47 testes automatizados passando build e typecheck OK

Repositório: inicializado em 07/09/2026 (4 commits de checkpoint)

Resumo executivo. Este relatório documenta, em linguagem técnica, o diagnóstico e a resolução de pendências do sistema *agenda Saúde*: desde o laudo original (que apontava 6 falhas críticas, 7 altas, 8 médias e cerca de 14 de baixa severidade) até a implantação, em 07/09/2026, das cinco pendências estruturais remanescentes (exclusão de conta no Supabase Auth, renovação de sessão, recuperação de senha, rate limit distribuído e trilha de auditoria), além de melhorias de produto (navegação, listagens, histórico, e-mail obrigatório do paciente e notificação por e-mail via Brevo). Cada correção é descrita com a **causa raiz**, o **impacto caso permanecesse**, os **arquivos alterados** e a **validação** executada. O documento lista ainda os commits realizados, as pendências remanescentes e as referências bibliográficas.

1. Objeto e contexto técnico

O *agenda Saúde* é um sistema web de agendamento de atendimentos voltado a unidades de saúde. A arquitetura é de três camadas lógicas com responsabilidades bem definidas:

Camada	Tecnologia	Responsabilidade
Frontend	React 18, TypeScript, Vite, Tailwind CSS e shadcn/ui	SPA com rotas do tipo <code>/dashboard/*</code> ; contrato de API centralizado em <code>services/</code>
Backend	Python 3.14, FastAPI, Pydantic v2	API REST em camadas controller → service → repository, validação e regras de negócio
Dados/Auth	Supabase (PostgreSQL + Auth + Storage)	Persistência (tabelas <code>usuarios</code> , <code>pacientes</code> , <code>agendamentos</code> etc.), autenticação por JWT

O Backend implementa padrão em camadas (controllers sem `try/except`, handler global de erros, repositórios sobre o cliente *supabase-py*) e o Frontend centraliza chamadas autenticadas. As pendências estruturais foram validadas com **pytest** (fakes, sem rede) e com **testes de integração E2E** executados contra o ambiente Supabase real, cobrindo os fluxos HTTP ponta a ponta.

2. Situação recebida (laudo de 13/08/2026)

O sistema foi entregue com problemas que o tornavam **não funcional de ponta a ponta**. A tabela resume o laudo original; os itens da revisão de 07/09/2026 são detalhados na seção 3.

2.1. Classificação por severidade

Severidade	Quant.	Exemplos representativos
Crítica	6	Cadastro quebrado (método inexistente → HTTP 500); rotas de dados sem autenticação; página de consultas inoperante (contrato divergente e variáveis inexistentes); token de login nunca salvo; vazamento de usuários em <code>/usuarios/me</code>
Alta	7	Sem proteção de rota/logout no frontend; fuso horário inconsistente; <code>IndexError</code> em respostas vazias; login 500 sem linha em <code>usuarios</code> ; exclusão não removia do Supabase Auth
Média	8	<code>exclude</code> malformado em repositório; CORS fixo; 403 no lugar de 401; formatos de erro divergentes; duplicação de diálogo de cancelamento
Baixa	≈14	<code>console.log</code> com dados sensíveis; código morto; documentação desatualizada; ausência de testes e de typecheck

2.2. Correções já implantadas antes de 07/09/2026 (resumo)

ID	Problema	Correção aplicada
----	----------	-------------------

C-1	<code>AuthService.register</code> inexistente	Implementado <code>register</code> com validação e sincronização com a tabela <code>usuarios</code>
C-2	Rotas de dados sem autenticação	Todas protegidas com <code>Depends(get_current_user)</code> → 401
C-3/C-4	Página de consultas quebrada / contrato divergente	Diálogo órfão removido; mapeamento único <code>mapAgendamento()</code> ; <code>tsc</code> no build
C-5	Token nunca salvo	Login passa a salvar <code>access_token</code>
C-6	<code>get_me</code> vazava a lista de usuários	<code>buscar_por_id</code> com <code>.eq()</code> + verificação de posse
A-1...A-5/A-7, M-1...M-8, L-1...L-14	Itens de severidade alta/média/baixa do laudo	Guard de rota e logout; UTC de ponta a ponta; acesso seguro a <code>response.data</code> ; CORS por env; soft delete; rate limit inicial; 17 testes; docs reescritas

Detalhamento completo consta em [RELATORIO_TECNICO.md](#) e [RELATORIO_COMPLETO_AGENDA_SAUDE_FABIANO_SANTOS.md](#).

3. Pendências resolvidas em 07/09/2026 (causa, impacto e correção)

As cinco pendências estruturais documentadas na seção 13 do relatório completo foram implementadas e validadas. Para cada item: **causa raiz** → **o que ocorreria se permanecesse** → **correção** → **arquivos**.

3.1. P1 — Exclusão de conta não removia o usuário do Supabase Auth (A-6)

Causa raiz. `DELETE /usuarios/{id}` removia apenas a linha da tabela `usuarios`, sem chamar `admin.delete_user()`. Como o Supabase Auth exige a `service role key`, o fluxo foi omitido por segurança, deixando **contas fantasmas**.

Impacto se permanecesse. Contas órfãs no Auth (impossibilitadas de logar, mas ainda existentes); descumprimento do direito de eliminação de dados pessoais (LGPD, art. 18); superfície de ataque e inconsistência entre Auth e tabela.

Correção. Criado cliente privilegiado `supabase_admin` quando `SUPABASE_SERVICE_ROLE_KEY` está presente; `UsuarioService.delete()` chama `admin.delete_user()` (tolerando "user not found") antes de remover a linha. Sem a chave, a exclusão retorna 503 explícito. Adicionado **guard**: usuário com agendamentos vinculados não é excluído (400 com mensagem clara), evitando violação de chave estrangeira.

Arquivos. `src/config/database.py` ; `src/services/usuario_service.py` ; `src/repositories/usuario_repository.py` ; `tests/services/test_usuario_service.py` (novo); `.env.example`.

3.2. P2 — Sessão sem renovação automática (refresh token)

Causa raiz. Não havia endpoint `/auth/refresh` e o frontend descartava o `refresh_token` retornado no login; chamadas usavam `fetch` espalhado com `Authorization` manual.

Impacto se permanecesse. JWT expira (~1 h) e o usuário é forçado a relogar no meio do uso, com perda de contexto; má experiência e interrupção de fluxos de atendimento.

Correção. Endpoint `POST /auth/refresh` no backend. No frontend, criado `services/apiClient.ts` (`apiFetch`) que injeta o token, e em resposta 401 renova a sessão com **single-flight** e repete a requisição uma vez. Todas as páginas autenticadas e o `agendamentoService` migrados para `apiFetch`; login persiste `refresh_token`; logout/401 limpam `token`, `refresh_token` e `usuario`.

Arquivos. `src/controllers/auth_controller.py` ; `src/services/auth_service.py` ; `services/apiClient.ts` (novo); `services/session.ts` ; `services/agendamentoService.ts` ; páginas autenticadas (`LoginPage`, `DashboardLayout`, `PlannerPage`, `AppointmentsPage`, `NewAppointmentPage`, `ReschedulePage`, `NewPatientPage`, `EditPatientPage`, `NewAgenteComunitarioPage`); testes de `auth_service`.

3.3. P3 — Recuperação de senha inexistente

Causa raiz. Não havia fluxo "esqueci minha senha" no backend nem no frontend.

Impacto se permanecesse. Usuário sem acesso ficava dependente de intervenção manual; risco de contas inativas e retrabalho administrativo.

Correção. `POST /auth/forgot-password` (dispara e-mail do Supabase e **sempre** responde sucesso, não revelando se o e-mail existe) e `POST /auth/reset-password` (valida os tokens do link em cliente isolado e troca a senha). Frontend: páginas `/forgot-password` e `/reset-password` (lê `#access_token` / `#refresh_token` do hash), link no login e aviso após reset. Redirect URLs configuradas no Supabase (local + produção).

Arquivos. `auth_service.py`; `auth_controller.py`; `ForgotPasswordPage.tsx` (novo); `ResetPasswordPage.tsx` (novo); `routes.tsx`; `LoginPage.tsx`; testes de `auth_service`.

3.4. P4 — Trilha de auditoria de agendamentos

Causa raiz. O soft delete (`cancelado`) registrava apenas `atualizado_em`, sem identificar *quem* criou/alterou/cancelou nem os dados da operação.

Impacto se permanecesse. Sem rastreabilidade de alterações em dados de saúde; dificuldade de responder a solicitações do titular (LGPD) e a auditorias da unidade.

Correção. Tabela `agendamentos_auditoria` (migração `sql/migracao_auditoria.sql`) com `usuario_id`, `acao` (`criado`|`atualizado`|`cancelado`), `dados` (JSONB) e `criado_em`. `AgendamentoService` registra a ação em `criar`, `atualizar` e `deletar` recebendo o autor via controller; falha de auditoria **não** interrompe o fluxo. Leitura via `GET /agendamentos/{id}/auditoria`, com resolução do nome do autor (2 consultas a `usuarios`). Migração executada e verificada no banco real.

Bugs descobertos no teste E2E (corrigidos). (a) leitura retornava vazio porque a query fazia `embed usuarios(nome,email)` sem existir FK PostgREST → removido o `embed` e resolvido nome por consulta; (b) exclusão de usuário com agendamentos gerava HTTP 500 por violação de FK → convertido em guard com HTTP 400 claro.

Arquivos. `sql/migracao_auditoria.sql` (novo); `repositories/agendamento_auditoria_repository.py` (novo); `services/agendamento_service.py`; `controllers/agendamento_controller.py`; `repositories/usuario_repository.py`; testes de agendamento.

3.5. P5 — Rate limit em memória não escala em multi-instância

Causa raiz. `MemoryRateLimiter` (5 req/min/IP) era por processo.

Impacto se permanecesse. Com N instâncias do backend, o mesmo IP poderia efetuar até 5×N tentativas; enfraquecimento do controle de força bruta em `/auth/login` e `/auth/register`.

Correção. `RedisRateLimiter` (janela fixa com `INCR + EXPIRE`) ativado quando `REDIS_URL` está definida; **fallback** automático para memória se o Redis estiver indisponível (resiliência). Dependência `redis==5.2.1` adicionada.

Arquivos. `src/utils/rate_limit.py`; `requirements.txt`; `tests/test_rate_limit.py` (novo).

3.6. Melhorias de produto implantadas (mesma sessão)

Item	Causa / motivação	Implementação	Arquivos
Menu de navegação	<code>DashboardLayout</code> não expunha links; o usuário ficava preso ao Planner (telas "órfãs", acessíveis só por URL)	Barra de navegação com <code>NavLink</code> (Planner, Consultas, Novo Agendamento, Pacientes, Agentes), destaque do item ativo e <code>end</code> na rota índice	<code>components/DashboardLayout.tsx</code>
Listagem de Pacientes	Existiam apenas telas de criação/edição	Tabela com filtros (nome, nº SUS, status), nome do agente vinculado e ação "Editar"	<code>pages/PacientesPage.tsx</code> (novo); <code>services/pacienteService.ts</code> (novo); <code>routes.tsx</code>
Listagem de Agentes	Sem tela de consulta/renomeio	Tabela com nº de pacientes vinculados, busca e diálogo "Editar nome" (PUT)	<code>pages/AgentesComunitariosPage.tsx</code> (novo); <code>services/agenteService.ts</code> (novo); <code>routes.tsx</code>
Histórico (UI de auditoria)	Endpoint existia, sem visualização	Botão "Histórico" por linha em Consultas + diálogo com ações (criação/alteração/cancelamento), data/hora, autor e horário registrado	<code>pages/AppointmentsPage.tsx</code> ; <code>services/agendamentoService.ts</code> ; <code>types/agendamento.ts</code>
E-mail obrigatório do paciente	Campo existia, mas opcional e sem validação amigável	Backend: <code>email: EmailStr</code> obrigatório no <code>PacienteCreate</code> . Frontend: <code>required</code> , regex e mensagem de erro inline (borda vermelha)	<code>schemas/paciente_schema.py</code> ; <code>NewPatientPage.tsx</code> ; <code>EditPatientPage.tsx</code>
Notificação por e-mail (Brevo)	Confirmar agendamento ao paciente	<code>NotificacaoService</code> (endpoint transacional <code>/v3/smtp/email</code> , chave via env); gatilho na criação do agendamento quando o paciente tem e-mail; falha não interrompe o fluxo; horário convertido para <code>America/Sao_Paulo</code>	<code>services/notificacao_service.py</code> (novo); <code>services/agendamento_service.py</code> ; <code>scripts/testar_brevo.py</code> (novo); <code>.env.example</code>

4. Arquivos alterados e criados

4.1. Backend — criados (novos)

Arquivo	Propósito
<code>src/repositories/agendamento_auditoria_repository.py</code>	Persistência e leitura da trilha de auditoria
<code>src/services/notificacao_service.py</code>	Envio transacional via Brevo (confirmar agendamento)
<code>sql/migracao_auditoria.sql</code>	DDL da tabela <code>agendamentos_auditoria</code> + índices
<code>tests/services/test_usuario_service.py</code>	Testes da exclusão com Supabase Auth (A-6) e guard
<code>tests/test_rate_limit.py</code>	Testes dos limitadores (memória e Redis com fake)
<code>tests/test_notificacao_service.py</code>	Testes do payload Brevo e resiliência
<code>scripts/verificar_migracoes.py</code>	Verifica se as migrações SQL foram aplicadas
<code>scripts/testar_brevo.py</code>	Envia e-mail de teste via Brevo

4.2. Backend — alterados

Arquivo	Alteração
<code>src/config/database.py</code>	Cliente <code>supabase_admin</code> (service role) condicional
<code>src/controllers/auth_controller.py</code>	Endpoints <code>/auth/refresh</code> , <code>/forgot-password</code> , <code>/reset-password</code> + DTOs
<code>src/services/auth_service.py</code>	Métodos <code>refresh</code> , <code>forgot_password</code> , <code>reset_password</code>
<code>src/services/usuario_service.py</code>	Delete remove do Auth; guard de agendamentos vinculados
<code>src/services/agendamento_service.py</code>	Recebe <code>usuario_id</code> ; registra auditoria; <code>listar_auditoria</code> ; notifica paciente
<code>src/controllers/agendamento_controller.py</code>	Passa o autor; rota <code>GET /{id}/auditoria</code>

src/repositories/usuario_repository.py	possui_agendamentos e buscar_nomes
src/utils/rate_limit.py	RedisRateLimiter + seleção por env
src/schemas/paciente_schema.py	email obrigatório no PacienteCreate
requirements.txt	redis==5.2.1
.env.example	Chaves de prod, Brevo, FRONTEND_URL , REDIS_URL
tests/services/test_auth_service.py e test_agendamento_service.py	Novos casos (refresh/reset/auditoria/notificação)

4.3. Frontend — criados e alterados

Arquivo	Alteração
services/apiClient.ts novo	apiFetch com injeção de token e refresh automático
services/pacienteService.ts , services/agenteService.ts novos	Contratos de listagem e atualização
pages/ForgotPasswordPage.tsx , pages/ResetPasswordPage.tsx novos	Fluxo de recuperação de senha
pages/PacientesPage.tsx , pages/AgentesComunitariosPage.tsx novos	Listagens com filtros e ações
components/DashboardLayout.tsx	Menu de navegação com destaque da rota ativa
routes.tsx	Rotas de listagem e de recuperação de senha
services/agendamentoService.ts	Migrado a apiFetch ; fetchAgendamentoAuditoria
services/session.ts	Limpeza de refresh_token em 401/logout
pages/AppointmentsPage.tsx	Botão e diálogo de histórico (auditoria)
pages/LoginPage.tsx	Persistência de refresh_token ; link de recuperação; aviso pós-reset
pages/NewPatientPage.tsx , pages/EditPatientPage.tsx	E-mail obrigatório com validação visível
Páginas PlannerPage , NewAppointmentPage , ReschedulePage , NewAgenteComunitarioPage	Chamadas migradas de fetch para apiFetch (refresh/401)
types/agendamento.ts	Tipo RegistroAuditoria

4.4. Infraestrutura / documentação

Arquivo	Alteração
frontend/vercel.json novo	Rewrites do SPA (deep links/refresh não geram 404)
backend/.env.production.example novo	Referência de variáveis para produção
README.md	Seção Produção (Render + Vercel + Supabase) e envs
RELATORIO_TECNICO.md / RELATORIO_COMPLETO_*.md	Status das pendências atualizado (seção 13) e cobertura de testes

5. Validação executada

Verificação	Comando / método	Resultado
Testes unitários/integração (fakes)	pytest tests -q	47 passed (era 17; +30 novos casos)
Typecheck	tsc --noEmit (frontend)	OK (sem erros)
Build de produção	npm run build	OK (Vite, ~524 kB JS)
Smoke test da API	uvicorn + / , /health , rota sem token	200 / 200 / 401
Integração E2E real (Supabase)	Fluxos HTTP completos (usuário descartável)	OK — cadastro, login, refresh, CRUD de agendamento, auditoria, cancelamento

E2E confirmação de correções	Auditoria lida + guard A-6	13/13
E2E fluxo de reset de senha	Link de recovery via admin → reset → login com nova senha	OK (validação em 200; confirmação de login pós janela de rate limit)
Migrações no banco real	<code>scripts/verificar_migracoes.py</code> + consultas <code>information_schema</code>	aplicadas (<code>cancelado</code> , <code>timestampz</code> , <code>agendamentos_auditoria</code>)
Brevo (envio real)	<code>scripts/testar_brevo.py</code>	enviado (confirmação recebida no destinatário)

Obs.: o teste de integração final de "login com nova senha" foi executado em passo isolado retornando HTTP 200; a checagem de login dentro da mesma janela de 60 s esbarrou no rate limit (429) — comportamento esperado do próprio controle implantado (P5).

6. Commits realizados

O repositório local **não possuía histórico**; foi inicializado em 07/09/2026 com os quatro commits abaixo (trabalho de checkpoint após a validação). Nenhuma credencial ou segredo foi versionado (`.env*` , `node_modules` , `venv` e `dist` estão no `.gitignore`).

Hash	Autor	Data	Mensagem
b7299d5	Fabiano Santos	07/09/2026 22:04	feat(backend): refresh/reset de senha, A-6 delete no Auth, auditoria de agendamentos, rate limit Redis, notificacao Brevo e email obrigatorio de paciente
45344d9	Fabiano Santos	07/09/2026 22:04	feat(frontend): menu de navegacao, telas de listagem (pacientes/agentes), historico de auditoria, refresh de sessao automatico, recuperacao de senha e validacao de email do paciente
f430237	Fabiano Santos	07/09/2026 22:04	docs+infra: guia de producao (Vercel/Render), vercel.json SPA, envs de exemplo e relatorios atualizados
8826e9e	Fabiano Santos	07/09/2026 22:04	chore(backend): configuracoes do pytest

Mensagens seguem a convenção *Conventional Commits* (feat/fix/docs/chore/refactor/test) exigida nas diretrizes do projeto.

7. Pendências remanescentes

7.1. Para concluir a publicação (ambiente de produção)

Pendência	Detalhe / recomendação	Responsável
Push para o GitHub	Definir o <i>remote origin</i> e publicar a branch <code>main</code>	Equipe
Reativar e atualizar o backend no Render	Serviço <code>agendasaude-1r41.onrender.com</code> encontra-se suspenso e com código anterior; reativar, definir envs (incl. <code>SUPABASE_SERVICE_ROLE_KEY</code> , <code>FRONTEND_URL</code> , <code>CORS_ORIGINS</code>) e redeploy a partir do GitHub	Equipe / conta Render
Redeploy do frontend na Vercel	Republicar com o código atual; <code>VITE_API_URL</code> = URL pública do backend; <code>vercel.json</code> já cobre rotas SPA	Equipe / conta Vercel
Site URL no Supabase	Alterar <i>Site URL</i> de <code>http://localhost:5173</code> para o domínio de produção (redirects locais permanecem na allowlist)	Equipe
Remetente "oficial" no Brevo	Envio atual usa <code>fabianotextil@gmail.com</code> (validado). Para remeter como a unidade, verificar sender/domínio no Brevo e ajustar <code>BREVO_FROM_EMAIL</code>	Equipe

7.2. Evoluções recomendadas (não bloqueiam o uso)

- **Lembrete automático de consulta** (ex.: 24 h antes) — requer agendador/cron (celery, APScheduler ou função agendada no Supabase) sobre a mesma base do `NotificacaoService` ;
- **Exclusão/reatribuição de pacientes e agentes** com regras de integridade explícitas (hoje há somente listagem/edição);
- **Telas administrativas** (gestão de usuários/tipos) restritas a perfil adequado;
- **Segmentação de auditoria** para outras entidades (pacientes, usuários), não só agendamentos;
- Mensagens de e-mail parametrizáveis por unidade (assinatura/logotipo) e histórico de envios (webhooks/estatísticas do Brevo).

8. Considerações finais

A intervenção de 07/09/2026 encerrou as cinco pendências estruturais e ainda elevou a maturidade do produto com navegação, listagens, trilha de auditoria com interface, cadastro de paciente com e-mail obrigatório e notificação automática via Brevo. A suíte de testes saltou de 17 para **47 casos**; typecheck, build e testes de integração contra o Supabase real passaram, e um envio transacional real via Brevo foi confirmado. Os riscos residuais concentram-se na **finalização do deploy** (Render suspenso, push e variáveis de produção) e em evoluções opcionais listadas na seção 7.

9. Referências bibliográficas

1. SUPABASE. *Supabase Docs — Auth, Database, Row Level Security, Management API*. Disponível em: <https://supabase.com/docs>. Acesso em: 7 set. 2026.
2. SUPABASE. *supabase-py (Python Client Library v2)*. Disponível em: <https://supabase.com/docs/reference/python/introduction>. Acesso em: 7 set. 2026.
3. FASTAPI. *FastAPI Documentation — Security, Dependencies, Exception Handling*. Disponível em: <https://fastapi.tiangolo.com>. Acesso em: 7 set. 2026.
4. REMAIN, Sébastien. *Pydantic v2 — Data Validation and Settings Management*. Disponível em: <https://docs.pydantic.dev>. Acesso em: 7 set. 2026.
5. REACT. *React — Referência da API e Guia*; REACT ROUTER. *Data Routers e Navegação*. Disponível em: <https://react.dev> e <https://reactrouter.com>. Acesso em: 7 set. 2026.
6. VITE. *Vite — Frontend Tooling*; TAILWIND LABS. *Tailwind CSS*; SHADCN. *shadcn/ui*. Disponível em: <https://vite.dev>, <https://tailwindcss.com> e <https://ui.shadcn.com>. Acesso em: 7 set. 2026.
7. BREVO. *Brevo API v3 — Transactional Email (SMTP API)*. Disponível em: <https://developers.brevo.com/reference/sendtransacemail>. Acesso em: 7 set. 2026.
8. REDIS. *Commands INCR e EXPIRE; padrões de rate limiting*. Disponível em: <https://redis.io/commands> e <https://redis.io/glossary/rate-limiting>. Acesso em: 7 set. 2026.
9. OWASP FOUNDATION. *OWASP — Authentication Cheat Sheet; Rate Limiting*. Disponível em: <https://owasp.org>. Acesso em: 7 set. 2026.
10. BRASIL. *Lei nº 13.709, de 14 de agosto de 2018 (LGPD) — especialmente arts. 16 e 18 (eliminação de dados pessoais) e tratamento de dados de saúde*. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm. Acesso em: 7 set. 2026.
11. RFC 3339 / ISO 8601. *Date and Time on the Internet: Timestamps — uso de UTC e offsets*. IETF. Disponível em: <https://datatracker.ietf.org/doc/html/rfc3339>. Acesso em: 7 set. 2026.
12. RENDER. *Render Docs — Web Services e variáveis de ambiente*; VERCEL. *Vercel Docs — SPA rewrites e variáveis de ambiente*. Disponível em: <https://render.com/docs> e <https://vercel.com/docs>. Acesso em: 7 set. 2026.
13. DATE-FNS. *date-fns — formatação de datas*. Disponível em: <https://date-fns.org>. Acesso em: 7 set. 2026.

Documento gerado pela Equipe agenda Saúde — PI2. Ferramentas: revisão estática de código, testes automatizados (pytest), testes de integração contra Supabase e validação HTTP. Este arquivo não contém segredos (chaves de API não são impressas).